Procesadores del Lenguaje

Curso 2025-2026

Práctica Final – Segunda Parte

Traductor de un subconjunto de C a un lenguaje en notación prefija (Lisp)

Esta segunda parte de la práctica continúa ampliando las especificaciones para permitir la traducción de un pequeño subconjunto de código C. Esto incluirá estructuras condicionales (if, while, for), funciones y matrices. La salida seguirá estando en notación prefija, que corresponde a una variante del lenguaje Lisp, que utilizaremos como lenguaje intermedio.

En futuras partes también se abordará el diseño de un pequeño backend para traducir de Lisp a un lenguaje de bajo nivel para una máquina de pila (como alternativa a los cpus más convencionales programados con Risc V o lenguaje ensamblador x86).

Trabajo a realizar:

1. Leed el enunciado pasado completo antes de comenzar.
2. Revisad las especificaciones de éste documento con anotaciones añadidas resaltadas. Comprobad que vuestro trabajo ya realizado sigue dichas especificaciones.
3. Renombrad vuestro fichero de código en sucesivas sesiones a trad2.y, trad3.y, ...
4. Continuad con el trabajo en el punto que corresponda, desarrollando los puntos indicados en las especificaciones hasta donde os de tiempo. En futuras sesiones continuaremos con ellas.
5. Comprobad vuestra instalación local de CLisp.
6. Podéis evaluar los resultados de traducción usando el intérprete online https://rextester.com/l/common_lisp_online_compiler. Para ello podéis:
   a. editar un fichero (por ejemplo, `prueba.c`) con una serie de expresiones
   b. ejecutar `cat prueba.c | ./trad >prueba.l`
   c. copiar la salida y pegarla en la ventana del intérprete
   d. con F8 se compila y ejecuta.

Entrega de Sesión:
Seguid las instrucciones del entregador.

- Un fichero `trad2.y` con el trabajo realizado hasta el momento. Incluid en la cabecera del fichero dos líneas de comentario iniciales. La primera con vuestros número de equipo y nombres (separados por comas). Y la segunda con los correos electrónicos (separados con una coma).
- Un documento llamado `trad2.pdf` con una breve explicación del trabajo realizado.

Después de hacer la entrega, descargad vuestro fichero y comprobad que funciona correctamente y que cumple con las indicaciones.

Especificaciones (Frontend)

Las especificaciones previas del fichero **`trad1e.y`** proporcionado son:

- Hay una estructura jerárquica en la gramática que empieza con:
  - **`Axioma`**, se encarga de la recursividad (**`r_axioma`**) y deriva:
  - **`Sentencia`**, que a su vez deriva la asignaciones y las impresión de expresiones.
- La sentencia que contiene únicamente una expresión se ha eliminado. Aunque en el lenguaje C se permiten sentencias como **`1+2;`** no les sacaremos partido, y además pueden ser una fuente de conflictos.
- Podéis añadir vuestra solución para asignaciones encadenadas si disponéis de ella.
- El código de **`trad1e.y`** utiliza código diferido. Dejaremos el posible uso de AST para más adelante.
- Estudiad las soluciones proporcionadas para:
  a. **`termino ::= operando`**
  b. **`termino ::= +`** operando
  c. **`termino ::= - operando`**
  d. **`operando ::= IDENTIF`**
  e. **`operando ::= ( expresion )`**

Proponemos una secuencia de pasos para desarrollar esta práctica. Se recomienda abordar cada uno de los pasos de forma secuencial.

1. **Variables Globales**. Incluid la definición de variables globales en la gramática.
   a. La definición en C será **`int <id>;`**[2] que debe traducirse a **`(setq <id> 0)`**. Es necesario incluir un segundo parámetro en Lisp para inicializar las variables. En el caso de la inicialización más simple en C, este valor será 0 por omisión. Puede haber 0, 1 o más líneas con definiciones de variables cada una comenzando por **`int`**.
   b. Ampliad la gramática para contemplar la definición de una variable con inicialización incluida: **`int <id> = <cte> ;`** que debe ser traducido a **`(setq <id> <cte>)`**. Utilizamos aquí **`<cte>`** para representar un valor numérico constante. No consideraremos por ahora expresiones en las declaraciones.
   c. Ampliad la gramática para contemplar la definición múltiple de variables con asignaciones opcionales: **`int <id1> = 3, <id2>, …, <idk> = 1 ;`** que debe ser traducido a una secuencia de definiciones individuales **`(setq <id1> 3) (setq <id2> 0) … (setq <idk> 1)`** . El orden de impresión de las variables debe corresponder al de la declaración. Prestad atención a que se asignan valores constantes (numéricos), no expresiones evaluables. Esto es así porque para las variables globales en C no es posible evaluar dichas expresiones en tiempo de compilación.

   Esto corresponde a la definición de variables globales. Recordamos que en C no está permitido inicializar variables con expresiones (con variables y funciones) en la instrucción en que es declarada puesto que el proceso se realiza en tiempo de compilación. La evaluación de expresiones se hace en tiempo de ejecución. Algunos compiladores sí disponen de una fase previa capaz de evaluar expresiones simples del tipo **`int ab32 = 3+7 ;`** y de sustituirlas por **`int ab32=10;`** pero sigue siendo una tarea realizada en tiempo de compilación.

[2] A partir de este punto comenzamos a traducir de C a Lisp

2. **Función MAIN**. En C `main` es el procedimiento/función principal. Debe tener una palabra reservada en la tabla correspondiente, y debe enlazar con el *Token* `Main`. Ampliad la gramática para reconocer la función `main`. Debe permitir la inclusión de sentencias. Prestad atención al siguiente punto. Incluimos un ejemplo de traducción:

C	Lisp

`int a ;`
`main () {`
`  @ (a + 1) ;`
`}`
	`(setq a 0)`
	`(defun main ()`
	`  (print (+ a 1))`
	`)`

`//@ (main)`	`(main) ; Para ejecutar el programa`

A través de la gramática es posible obligar a que exista una (única) función `main`.

Considerad que la estructura de un programa en C debe ser:
`<Decl Variables> <Def Funciones>.` En teoría debería poder intercalarse ambos tipos de definiciones, pero eso puede producir conflictos en el *parser*. Por ello seguiremos una estructura fija. Las sentencias que define la gramática sólo deben aparecer dentro del cuerpo de una función. Revisad la estructura de vuestra gramática. Debe estar diseñada de forma muy cuidadosa y estructurada, intentando que sea lo más jerárquica posible.

Aquí se indican varias cuestiones.

1) Hay que diseñar una gramática lo más jerárquica o estructurada posible. Los nombres de No Terminales deben escogerse con cuidado y deben ser representativos y significativos.

2) Esto evitará la aparición de errores típicos de diseños "enmarañados" o excesivamente complicados.

3) Se sugiere emplear una estructura de programa que empiece con la declaración de variables y luego con la definición de funciones.

4) La recomendación es definir las funciones en orden inverso de jerarquía, empezando con las funciones más sencillas y terminando por la principal, el `main`. Esto permitirá que más adelante el traductor tenga siempre referencia de las funciones que se usan porque ya se han definido previamente. Si se definen primero las funciones principales y luego las de inferior jerarquía, un compilador necesitará hacer averiguaciones sobre el tipo de las funciones que se llaman y que aún no se han definido, o emplear dos pasadas para realizar traducciones parciales y condicionadas.

El compilador de C requiere en estos casos de una declaración previa (prototipo) de las funciones. En Lisp deben definirse las funciones antes de ser usadas. Para evitarnos complicaciones con los prototipos usaremos programas en C con las funciones en orden jerárquico inverso.

3. **Impresión de cadenas**. Para imprimir cadenas literales proponemos: **`puts(<string>);`** que se convierte en **`(print <string>)`**. Ejemplo: **`puts("Hola mundo");`** que se convierte en **`(print "Hola mundo")`**. Comprobad que **`yylex`** detecta las secuencias de entrada entre dobles comillas y las envía como un *token* **`String`**.

4. **Impresión de expresiones y cadenas**. Sustituid la sentencia para imprimir usando **`@`** por una nueva que comience con la palabra reservada **`printf`**. El formato de la impresión es **`printf(<string>, <elem1>, … , <elemN>);`**

   a. Comenzad por la versión simplificada **`printf(<string>, <elem1>);`** traduciéndola a **`(princ <elem1>)`**. Tomad nota que el contenido de la primera cadena, la cadena de formato, es complejo y requiere de un procesamiento muy elaborado para interpretarla que por ahora excede nuestros objetivos. Por ello debe ser reconocida por la gramática, pero no se traducirá de ninguna forma. Se omitirá en la traducción. El segundo parámetro **`<elem1>`** puede ser o bien una **`<expresion>`** o bien un **`<string>`**, y se traducirá con **`(princ <elem1>)`**

   b. Ampliad la gramática para reconocer:
   `printf(<string>, <elem1>, … , <elemN>);` El segundo y los siguientes parámetros **`<elemX>`** pueden ser o bien una **`<expresion>`** o bien un **`<string>`**, y se traducirán con **`(princ <elemX>)`** preservando el orden original. Es decir: **`(princ <elem1>)...(princ <elemN>)`**

   > La función **`print`** genera un salto de línea al final. La función **`princ`** omite dicho salto de línea y las dobles comillas de los strings..

5. **Operadores, precedencia y asociatividad**. Los operadores aritméticos, lógicos y de comparación en C se pueden combinar sin que el programador pueda apreciar matices particulares. En realidad se trata de operadores de diferente naturaleza. Los primeros devuelven valores numéricos, mientras que los lógicos y de comparación devuelven valores booleanos. Tendría sentido tratarlos por separado en la gramática, pero esto debe hacerse con cuidado dado que puede producir algunos conflictos o perderse las precedencias definidas por las directivas de bison. Los operadores lógicos y de comparación en Lisp se relacionan con los equivalentes en C en la siguiente tabla. Prestad atención a la asociatividad y precedencia definida para cada operador nuevo que incluyáis.

   https://en.cppreference.com/w/c/language/operator_precedence

C	&&	||	!	!=	==	<	<=	>	>=	%
Lisp	and	or	not	/=	=	<	<=	>	>=	mod

6. **Estructura de control WHILE**: **`while ( <expr> ) { <codigo> }`** . La traduciremos con la estructura **`(loop while <expr> do <codigo>)`** . Prestad atención a que esta sentencia de control en C no requiere de un separador ; al final.

   Y, ¿qué sucede si la entrada contiene algo como **`while(10-a){ a=a+1 ; }`** ? Más adelante explicaremos cómo tratar estos casos.

7. **Estructura de control IF**. Traducir la estructura **`if (<expr>) { <codigo> }`** con **`(if <expr> <codigo>)`** . En Lisp la estructura if sin else acaba de forma temprana con el último paréntesis. Ampliar la estructura de control if con el bloque else: **`if (<expr>) { <codigo1> } else { <codigo2> }`** se traduce como **`(if <expr> <codigo1> <codigo2>)`**. Esta ampliación puede provocar algún tipo de conflicto en el código generado por bison. Estudiad en las transparencias de teoría el origen de dicho conflicto, e intentad resolverlo. Se recomienda el uso de los marcadores de bloque { y } obligatorios. Recordad que la estructura de control **`if`** no termina en ; cuando se usan llaves.

   Planteamos un ejemplo de estructura mínima para el if-then-else:

C	Lisp

?	`(defun prueba-if (flag)`
	`  (if flag`
	`  123`
	`  456)`
	`)`

	`(print (prueba-if T))  ⇒ 123` `(print (prueba-if NIL)) ⇒ 456`

   Esto muestra el carácter funcional de Lisp, que no tiene correspondencia en C. En Lisp las expresiones devuelven un valor. En C dichos valores deben asignarse a una variable. No se podrá usar directamente la notación funcional de Lisp al traducir desde C.

   Ejemplo de traducción de C a Lisp:

C	Lisp

`int a = 1 ;`
`int b ;`
 	
`main () {`
`  if (a == 0) {`
`  b = 123 ;`
`  } else {`
`  b = 456 ;`
`  }`
`  printf (¨%d¨, b) ;`
`}`
`//@ (main)`
	`(setq a 1)`
	`(setq b 0)`
	
	`(defun main ()`
	`  (if (= 0 a)`
	`  (setq b 123)`
	`  (setq b 456)`
	`  )`
	
	`  (princ b)`
	`)`
	`(main)`

Para incluir en Lisp varias sentencias en las ramas then o else del if es necesario insertar una función **progn** que reciba dichas sentencias como parámetros. Esto se ilustra en el ejemplo de función **es_par**.

```
int ep ;

es_par (int v) {
  printf ("%d", v) ;
  if (v % 2 == 0) {
      puts (" es par") ;
      ep = 1 ;
  } else {
     puts (" es impar") ;
     ep = 0 ;
  }
  return ep ;
}
```

```
(setq ep 0)

(defun es_par (v)
   (princ v)
   (if (= (mod v 2) 0)
     (progn (print "es par")
            (setq ep 1))
     (progn (print "es impar")
            (setq ep 0)) )


    ep
}
```

En C las condicionales se definen como expresiones genéricas, sin hacer distinción de si se trata de expresiones aritméticas, booleanas o de comparación. Un motivo es que en C no existe el tipo booleano, se simula mediante valores enteros. El booleano False se representa con 0, y el True con cualquier otro valor. Un problema es que en Lisp no se admiten condicionales de tipo aritmético, provocan un error. La solución inicial que vamos a adoptar **inicialmente** es suponer que los programas de prueba usarán un C de tipo canónico, sin extravagancias como **while (10-a)** …

Ejemplos:

C	Lisp

`int a = 1 ;`
`main () {`
`  a = a && 0 ; // a = 0 ;`
`  while (10-a) { // 10-a != 0`
`  printf (¨%d¨, a) ;`
`  a = a + 1 ;`
`}`
	`(setq a 1)`
	`(defun main ()`
	`  (setq a (and a 0))`
	`  (loop while (- 10 a) do ; error`
	`  (princ a)`
	`  (setq a (- a 1))`
	`))`

`int a = 1 ;`
`main () {`
`  a = 0 ;`
`  while (a != 10) {`
`  printf (¨%d¨, a) ;`
`  a = a + 1 ;`
`  }`
`}`
	`(setq a 1)`
	`(defun main ()`
	`  (setq a 0)`
	`  (loop while (/= a 10) do`
	`  (princ a)`
	`  (setq a (+ a 1))`
	`  )`
	`)`

	`(main) ; Para ejecutar el programa`
Imprimirá la secuencia:	`0 1 2 3 4 5 6 7 8 9`

Se entiende que a priori no usaremos programas en C que contengan sentencias como las marcadas en Amarillo, deberán ser como las marcadas en Cyan.

8. **Variables Locales**. Dentro del cuerpo de las funciones pueden declararse variables, que tanto en C como en Lisp serán variables locales. Se emplea la misma traducción de las globales, pero en este caso deben generarse dentro del cuerpo de la función. Hay que tener cuidado a la hora de incluir la definición de variables locales en la gramática, para evitar conflictos con la definición de las globales.

   Revisad detenidamente el código del siguiente ejemplo para entender lo que se requiere en los siguientes apartados:

   a. Las variables globales se definirán como siempre usando **`setq`**
   b. Las variables locales se definirán usando **`setq`** con un cambio, concatenando su nombre con el nombre de la función. Esto se debe a que las variables definidas con **`setq`** son siempre variables globales en Lisp. El uso de la concatenación sirve para evitar colisiones entre variables locales y globales. Hasta el siguiente punto considerad sólo el caso de main
   c. Cambiad el código de salida de las sentencias de **asignación**: utilizad a partir de ahora **`setf`** en lugar de **`setq`**. Ambas funciones **`setq`** y **`setf`** son muy similares en Lisp, pero las utilizaremos para diferenciar entre declaraciones de variables con inicialización y sentencias de asignación.

C	Lisp

`int a ;`
`main () {`
`  int a = 4 ;`
`  a = a + 1 ;`
`  printf (“%d”, a+1) ;`
`}`
`//@ (main)`
	`(setq a 0)`
	`(defun main ()`
	`  (setq main_a 4)`
	`  (setf main_a (+ main_a 1))`
	`  (princ (+ main_a 1))`
	`)`
	`(main) ; Para ejecutar el programa`

   Utilizad el guión bajo _ para concatenar nombres en lugar del guión **-** . El guión es demasiado ambiguo con respecto al signo unario o al operador de resta.

   Para decidir qué variables utilizadas dentro de una función deben concatenarse con el nombre de la función, es necesario utilizar una tabla local en la que se insertan todas las variables declaradas localmente. Si una variable no se encuentra en esta tabla, puede deducirse que es una variable global y no debe concatenarse. Como punto de partida estudiad la función de búsqueda en la tabla de palabras reservadas.

9. **Estructura de Control FOR**. Implementación de la estructura de control `for` de C. Proponemos limitar el uso de este bucle a su versión más canónica. Esto es:

   `for ( <inicializ> ; <expr> ; <inc/dec>) { <codigo> }` donde debemos interpretar `<inicializ>` como una **sentencia de asignación de un valor a la variable índice**, `<expr>` es una expresión condicional cuyo valor lógico determina si se prosigue en el bucle, `<inc/dec>` representa la llamada a unos procedimientos predefinidos `INC(x)` o `DEC(x)`, que se encargan de incrementar o decrementar en una unidad la variable índice x. Para traducir la estructura `for` utilizad de nuevo la estructura `(loop while <expr> do <codigo>)`. Recordamos que en el caso de esta estructura la modificación de la variable índice se realiza en último lugar. La traducción a Lisp de las llamadas a `INC(x)` y `DEC(x)` será `(setq (x (+ x 1))` o `(setq x (- x 1))` o con los equivalentes `setf`.

   Ejemplo de bucle for en C con estilo canónico:

```
for (a = 0  ; a < n ; INC(a)) {
    . . .
}
```

Recordamos que en lenguaje C es posible usar esta estructura de control de forma menos estricta, incluyendo declaraciones y más de una sentencia en la inicialización, más operaciones simultáneas en la expresión condicional e incluso el cuerpo entero de código dentro del tercer campo de la cabecera. No obstante, desaconsejamos abordar estas opciones dado que no se van a contemplar e.

Los incrementos y decrementos de la variable índice se realizarán exclusivamente mediante los procedimientos INC y DEC.
Estos procedimientos no son nativos del lenguaje C, pero son fáciles de incorporar incluyendo las siguientes *macros* en el código C fuente:

```
#define INC(x) x=x+1
#define DEC(x) x=x-1
```

Esto permitirá que los ejemplos de prueba funcionen al compilar en C, pero no habrá interferencia con el traductor que ignora los #define

No implementéis operaciones del tipo `i++, ++i, i+=1`.

10. **Estructura de control Switch/Case**. Esta estructura se puede representar en Lisp de forma casi equivalente usando la estructura **`case`**.

    a) La sintaxis en C para un caso básico con cada **`case`** terminado en **`break`** es;

```
switch (<var>)  {
   case <val1> : <sentences> ; break ;
   case <val2> : <sentences> ; break ;
. . .
   default : <sentences> ; break ;
}
```

    que en Lisp se representaría con:

```
(case <var>
   (<val1>     <expressions>)
   (<val2>     <expressions>)
. . .
   (otherwise  <expressions>)
)
```

    tanto en C como en Lisp el uso de default y otherwise es opcional.

11. **Funciones**. En este apartado vamos a dar una serie de indicaciones para evitar problemas. Inicialmente proponemos el uso de las funciones sin tener en cuenta:
    a. Cómo opción inicial se propone la definición de funciones:
        i. **sin el valor de retorno**. En C podemos escribir funciones que no retornan nada, incluso aunque tengan definido un tipo (podemos considerarlo una mala praxis). Incluso aunque tengan un retorno, en la función que las llama se puede ignorar dicho valor. Eso sería el uso como procedimientos.
        ii. **sin tipos explícitos para las funcione**s. Si definimos en C una función sin tipo de retorno explícito se asigna de forma implícita el tipo `int`. En nuestro caso, esta opción es útil, puesto que de otra forma la definición de funciones y variables tendría la misma secuencia inicial de símbolos: <tipo> <identificador>. Esta circunstancia puede provocar conflictos, que con pueden solucionar con una reescritura adecuada de la gramática, pero supone una complicación.
        iii. **sin parámetros**. Esto servirá para hacer pruebas iniciales.
    b. Posteriormente podremos ampliar para que la función contenga:
        i. **un único argumento**. Se entiende que debe contemplar cero o un argumento. La gramática debe ampliarse tanto en la definición de las funciones como en las llamadas.
        ii. **una lista de argumentos**, que puede incluir cero, uno o más parámetros. La gramática debe ampliarse tanto en la definición de las funciones como en las llamadas. Hay que prestar atención a que la secuencia traducida de parámetros siga el mismo orden en ambos casos.
    c. **Añadimos el Retorno**. Recordamos la cuestión de los retornos estructurados.
        i. Una función bien estructurada sólo debería tener return como la última sentencia de la función: **`mifuncion () { … return <expresion>; }`** deberá traducirse a Lisp como:
        `(defun mifuncion ()  … <expresion> )` Conviene recordar que return en C va seguido de una expresión que no necesariamente lleva paréntesis. No la consideramos como una función, por lo que no requiere que se le pase el valor de retorno como argumento con paréntesis.
        ii. Sin embargo, en C se pueden situar en cualquier otro punto de la función, aunque sea una mala praxis dentro de la programación bien estructurada. Para ello, tendremos que traducir la sentencia
        `return expresion ;` como
        `(return-from <nombre-funcion> <expresion>)`.
    d. **Llamadas a funciones como procedimientos**. Recordamos que en C las funciones se pueden usar tanto como procedimientos, ignorando el valor que devuelve, o como funciones. En el primer caso deberíamos considerarlo una mala praxis si la función devuelve algún valor. Pero lo podemos contemplar también como un apartado adicional. Es decir, podremos considerar que las funciones de usuario además de intervenir en una expresión, como parámetro a la llamada a otra función, o en una asignación, también podrá ser una sentencia en la que sólo se llame a la función.

Ejemplos de funciones con un parámetro y retorno:

```c
#include <stdio.h>

square (int v) {
    return (v*v) ;
}

fact (int n) {
   int f ;
   if (n == 1) {
      f = 1 ;
   } else {
     f = n * fact (n-1) ;
   }
   return f ;
}

is_even (int v) {
   int ep  ;
   printf ("%d", v) ;
   if (v % 2 == 0) {
      puts (" is even") ;
      ep = 1 ;
   } else {
      puts (" is odd") ;
      ep = 0 ;
   }
   return ep ;
}

main () {
   printf ("%d\n", square (7)) ;
   puts (“ “) ;
   printf ("%d\n", fact (7)) ;
   puts (“ “) ;
   printf ("%d\n", is_even (7)) ;
   puts (“ “) ;
   printf ("%d\n", is_even (8)) ;
   puts (“ “) ;
   is_even (8) ;
}
//@ (main)
```

```lisp
(defun square (v)
    (* v v)
)

(defun fact (n)
     (setq fact_f 0)
     (if (= n 1)
        (setf fact_f 1)
        (setf fact_f (* n (fact (- n 1))))
      )
      fact_f
)


(defun is_even (v)
   (setq is_even_ep 0)
   (princ v)
   (if (= (mod v 2) 0)
      (progn
        (print " is even")
        (setf is_even_ep 1))
      (progn
        (print " is odd")
        (setf is_even_ep 0)) )
   is_even_ep
)

(defun main ()
    (princ (square 7))
    (print " ")
    (princ (fact 7))
    (print " ")
    (princ (is_even 7))
    (print " ")
    (princ (is_even 8))
    (print " ")
    (is_even 8)
)
(main)
```

```
⇒ 49
⇒ 5040
⇒ 7 is odd 0
⇒ 8 is even 1
⇒ 8 is even
```

12. **Implementación de Vectores.**

a. **Declaración**. Para declarar un Vector podemos usar una sintaxis de Lisp extendiendo la que usamos para variables: si en C definimos:
`int myvector[32];` lo traduciremos a `(setq myvector (make-array 32))` creando un vector llamado `myvector` con 32 elementos. **No contemplaremos la opción de inicializar los elementos, cómo sí se puede hacer con variables simples.** Tanto en C como en Lisp el primer elemento se indexará con 0. El tamaño del vector será un valor entero.

b. **Operando**. Para acceder a un elemento de un vector, para operar con él dentro de una expresión utilizaremos la función `aref`. Por ejemplo, si queremos imprimir el quinto elemento del vector `myvector` previamente definido: `printf("%s", myvector[4]);` deberá ser traducido como `(princ (aref myvector 4))`. La función `aref` usa dos parámetros, siendo el primero el vector, y el segundo el índice para acceder y devolver el valor indexado. El índice podrá ser una expresión.

c. **Receptor**. Para modificar un elemento de un vector, en la parte izquierda de una asignación `myvector[5]=123;` debemos usar: `(setf (aref mivector 5) 123)`. El índice podrá ser una expresión.

d. Podrán declararse como variables globales o locales. Excluimos la declaración de vectores como parámetros de una función. Es decir, NO contemplamos una declaración en C como: `int mifunc (int vect [7]) { … }`. A la hora de llamar a una función SI podemos pasarle un elemento indexado de un vector, se comportará como una variable individual: `a = max (v[i], v[i+1])`.

Mostramos ahora una secuencia de instrucciones Lisp para que se puedan valorar mejor las acciones y sus resultados.

Lisp	Resultado
`(setq b (make-array 5))`	
`(print b)`	`#(nil nil nil nil nil)`

`(setq i 0)`
`(loop while (< i 5) do`
`(setf (aref b i) i)`
`(setf i (+ i 1))`
`)`	

`(print b)`	`#(0 1 2 3 4)`
`(setf (aref b 0) 123)`	
`(print (aref b 0))`	`123`
`(print b)`	`#(123 1 2 3 4)`
`(print (aref b 10))`	`Aref Index out of range`